



Using ModelSim to Simulate Logic Circuits in Verilog Designs

For Quartus II 12.1

1 Introduction

This tutorial is a basic introduction to ModelSim, a Mentor Graphics simulation tool for logic circuits. We show how to perform functional and timing simulations of logic circuits implemented by using Quartus II CAD software.

The reader is expected to have the basic knowledge of the Verilog hardware description language, and the Altera Quartus® II CAD software.

Contents:

- Introduction to simulation
- What is ModelSim?
- Functional simulation using ModelSim
- Timing simulation using ModelSim

2 Background

Designers of digital systems are inevitably faced with the task of testing their designs. Each design can be composed of many modules, each of which has to be tested in isolation and then integrated into a design when it operates correctly.

To verify that a design operates correctly we use simulation, which is a process of testing the design by applying inputs to a circuit and observing its behavior. The output of a simulation is a set of waveforms that show how a circuit behaves based on a given sequence of inputs. The general flow of a simulation is shown in Figure 1.

There are two main types of simulation: functional and timing simulation. The functional simulation tests the logical operation of a circuit without accounting for delays in the circuit. Signals are propagated through the circuit using logic and wiring delays of zero. This simulation is fast and useful for checking the fundamental correctness of the designed circuit.

The second step of the simulation process is the timing simulation. It is a more complex type of simulation, where logic components and wires take some time to respond to input stimuli. In addition to testing the logical operation of the circuit, it shows the timing of signals in the circuit. This type of simulation is more realistic than the functional simulation; however, it takes longer to perform.

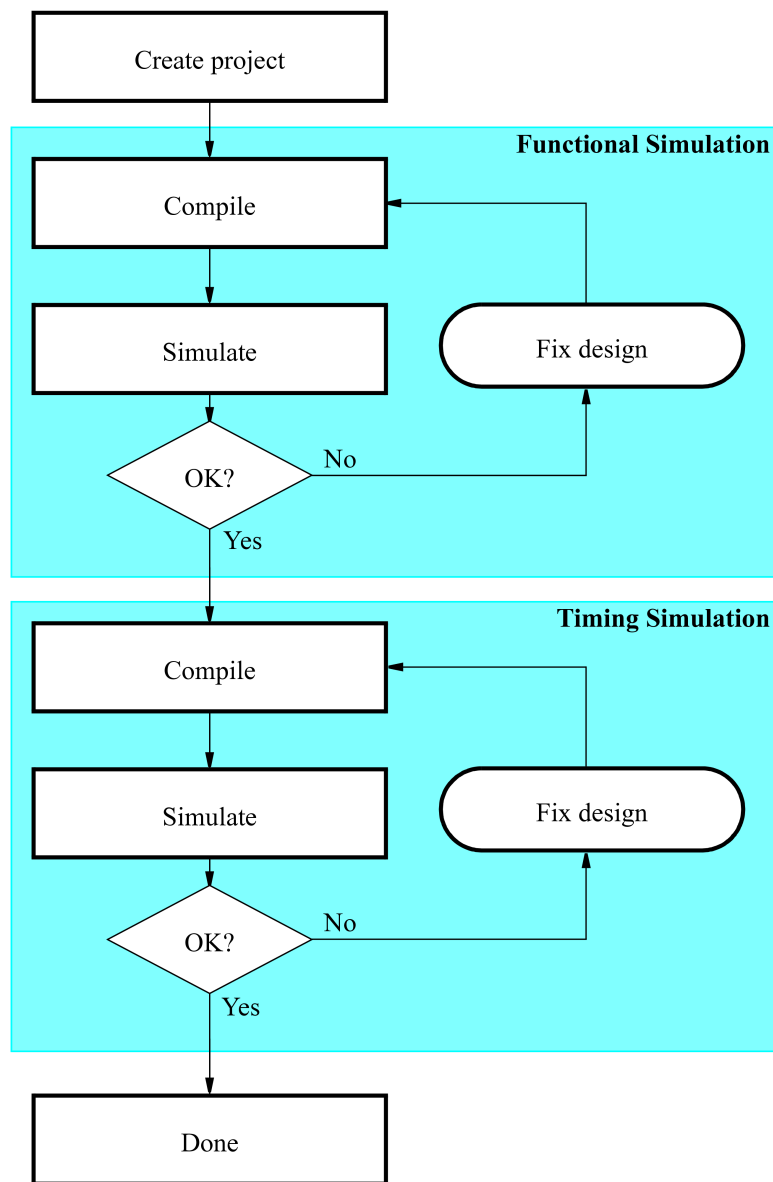


Figure 1. The simulation flow.

In this tutorial, we show how to simulate circuits using ModelSim. You will need the Quartus II CAD software and the ModelSim software, or ModelSim-Altera software that comes with Quartus II, to work through the tutorial.

3 Example Design

Our example design is a serial adder. It takes 8-bit inputs A and B and adds them in a serial fashion when the *start* input is set to 1. The result of the operation is stored in a 9-bit *sum* register.

A block diagram of the circuit is shown in Figure 2. It consists of three shift registers, a full adder, a flip-flop to store carry-out signal from the full adder, and a finite state machine (FSM). The shift registers A and B are loaded with the values of A and B . After the *start* signal is set high, these registers are shifted right one bit at a time. At the same time the least-significant bits of A and B are added and the result is stored into the shift register *sum*. Once all bits of A and B have been added, the circuit stops and displays the *sum* until a new addition is requested.

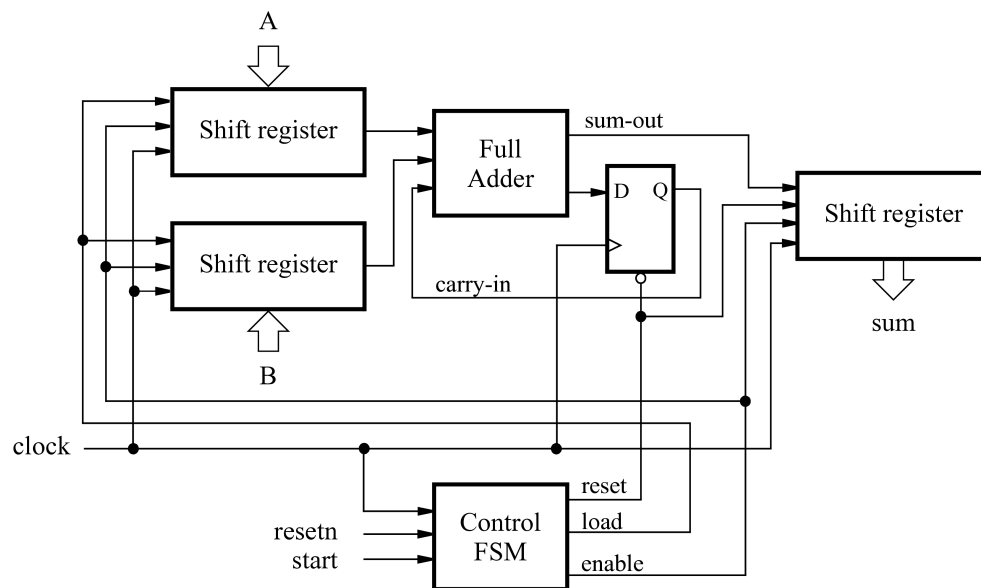


Figure 2. Block diagram of a serial-adder circuit.

The Verilog code for the top-level module of this design is shown in Figure 3. It consists of the instances of the shift registers, an adder, and a finite state machine (FSM) to control this design.

```

1.  module serial(A, B, start, resetn, clock, sum);
2.      input [7:0] A, B;
3.      input resetn, start, clock;
4.      output [8:0] LEDR;
5.
6.      // Registers
7.      wire [7:0] A_reg,B_reg;
8.      wire [8:0] sum;
9.      reg cin;
10.
11.     // Wires
12.     wire reset, enable, load;
13.     wire bit_sum, bit_carry;
14.
15.     // Confrol FSM
16.     FSM my_control(start, clock, resetn, reset, enable, load);
17.
18.     // Datapath
19.     shift_reg reg_A( clock, 1'b0, A, 1'b0, enable, load, A_reg);
20.     shift_reg reg_B( clock, 1'b0, B, 1'b0, enable, load, B_reg);
21.
22.     // a full adder
23.     assign bit_carry, bit_sum = A_reg[0] + B_reg[0] + cin;
24.
25.     always @(posedge clock)
26.     begin
27.         if (enable)
28.             if (reset)
29.                 cin <= 1'b0;
30.             else
31.                 cin <= bit_carry;
32.     end
33.
34.     shift_reg reg_sum( clock, reset, 9'd0, bit_sum, enable, 1'b0, sum);
35.     defparam reg_sum.n = 9;
36. endmodule

```

Figure 3. Verilog code for the top-level module of the serial adder.

The Verilog code for the FSM is shown in Figure 4. The FSM is a 3-state Mealy finite state machine, where the first and the third state waits for the *start* input to be set to 1 or 0, respectively. The computation of the sum of *A* and *B*

happens during the second state, called WORK_STATE. The FSM completes computation when the counter reaches a value of 8, indicating that inputs *A* and *B* have been added. The state diagram for the FSM is shown in Figure 5.

```

1.  module FSM(start, clock, resetn, reset, enable, load);
2.      parameter WAIT_STATE = 2'b00, WORK_STATE = 2'b01, END_STATE = 2'b11;
3.      input start, clock, resetn;
4.      output reset, enable, load;
5.
6.      reg [1:0] current_state, next_state;
7.      reg [3:0] counter;
8.
9.      // next state logic
10.     always@(*)
11.     begin
12.         case(current_state)
13.             WAIT_STATE:
14.                 if (start) next_state <= WORK_STATE;
15.                 else next_state <= WAIT_STATE;
16.             WORK_STATE:
17.                 if (counter == 4'd8) next_state <= END_STATE;
18.                 else next_state <= WORK_STATE;
19.             END_STATE:
20.                 if (~start) next_state <= WAIT_STATE;
21.                 else next_state <= END_STATE;
22.             default: next_state <= 2'bxx;
23.         endcase
24.     end
25.
26.     // state registers and a counter
27.     always@(posedge clock or negedge resetn)
28.     begin
29.         if (~resetn)
30.         begin
31.             current_state <= WAIT_STATE;
32.             counter = 'd0;
33.         end
34.     else
35.     begin

```

Figure 4. Verilog code for the FSM to control the serial adder (Part *a*).

```

36.          current_state <= next_state;
37.          if (current_state == WAIT_STATE)
38.              counter <= 'd0;
39.          else if (current_state == WORK_STATE)
40.              counter <= counter + 1'b1;
41.          end
42.      end
43.      // Outputs
44.      assign reset = (current_state == WAIT_STATE) & start;
45.      assign load = (current_state == WAIT_STATE) & start;
46.      assign enable = load | (current_state == WORK_STATE);
47.  endmodule

```

Figure 4. Verilog code for the FSM to control the serial adder (Part b).

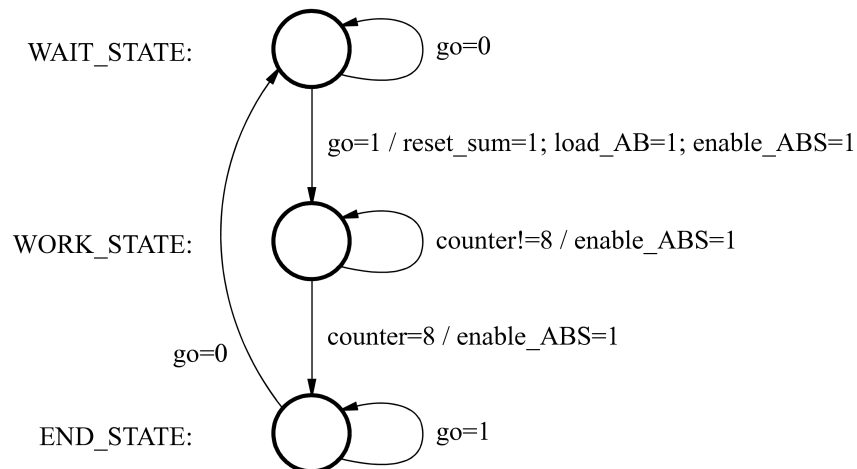


Figure 5. State diagram.

The Verilog code for the shift register is given in Figure 6. It consists of synchronous control signals to allow data to be loaded into the shift register, or reset to 0. When enable input is set to 1 and the data is not being loaded or reset, the contents of the shift register are moved one bit to the right (towards the least-significant bit).

```
1.  module shift_reg( clock, reset, data, bit_in, enable, load, q);
2.      parameter n = 8;
3.
4.      input clock, reset, bit_in, enable, load;
5.      input [n-1:0] data;
6.      output reg [n-1:0] q;
7.
8.      always@(posedge clock)
9.      begin
10.         if (enable)
11.             if (reset)
12.                 q <= 'd0;
13.             else
14.                 begin
15.                     if (load)
16.                         q <= data;
17.                     else
18.                         begin
19.                             q[n-2:0] <= q[n-1:1];
20.                             q[n-1] <= bit_in;
21.                         end
22.                     end
23.                 end
24.      endmodule
```

Figure 6. Verilog code for the shift register.

The design is located in the *example/functional* and *example/timing* subdirectories provided with this tutorial. A Quartus II project for this design has been created as well.

In the following sections, we use the serial adder example to demonstrate how to perform simulation using ModelSim. We begin by describing a procedure to perform a functional simulation, and then discuss how to perform a timing simulation.

4 Functional Simulation with ModelSim

We begin this tutorial by showing how to perform a functional simulation of the example design. We start by opening the ModelSim program.

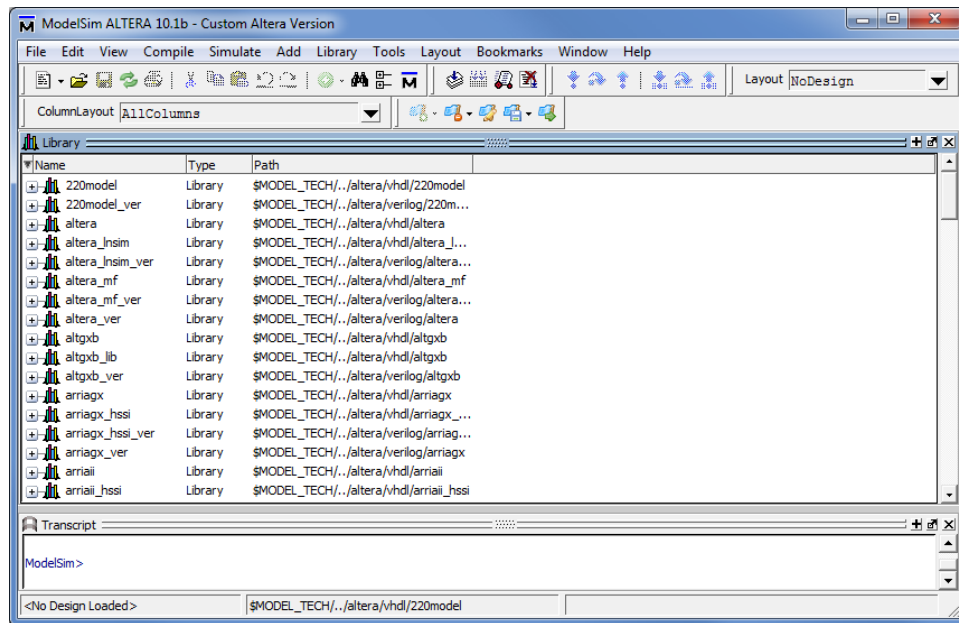


Figure 7. ModelSim window.

The ModelSim program window, shown in Figure 7, consists of three sections: the main menu at the top, a set of workspace tabs, and a command prompt at the bottom. The menu is used to access functions available in ModelSim. The workspace contains a list of modules and libraries of modules available to you, as well as details of the project you are working on. A new work area will appear on the right of the libraries of modules when needed to display waveforms and/or text files. Finally, the command prompt at the bottom shows feedback from the simulation tool and allows users to enter commands.

To perform simulation with ModelSim follow a basic flow shown in Figure 1. We begin by creating a project where all design files to be simulated are included. We compile the design and then run the simulation. Based on the results of the simulation, the design can be altered until it meets the desired specifications.

4.1 Creating a Project

To create a project in ModelSim, select File > New > Project.... A Create Project window shown in Figure 8 will appear.

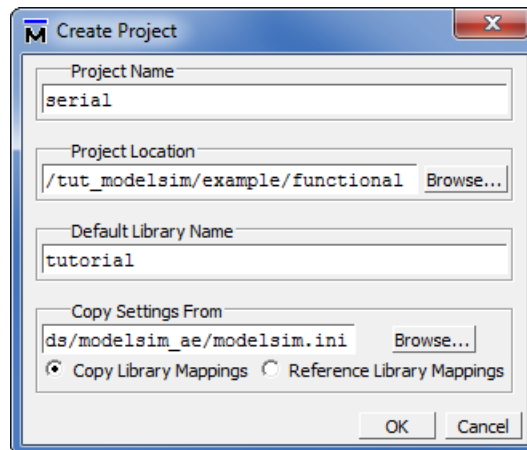


Figure 8. Creating a new project.

The create project window consists of several fields: project name, project location, default library name, and copy settings field. Project name is a user selected name and the location is the directory where the source files are located. For our example, we choose the project name to be *serial*, to match the top-level module name of our example design, and the location of the project is the *example/functional* subdirectory.

The default library name field specifies a name by which ModelSim catalogues designs. For example, a set of files that describe the logical behaviour of components in an Altera Cyclone II device are stored in the *cycloneii* library. This allows the simulator to include a set of files in simulation as libraries rather than individual files, which is particularly useful for timing simulations where device-specific data is required. For the purpose of this tutorial, specify *tutorial* as the library name for your project.

The last field in the create project window is the copy settings field. This allows default settings to be copied from the initialization file and applied to your project. Now, click OK to proceed to add files to the project using the window shown in Figure 9.

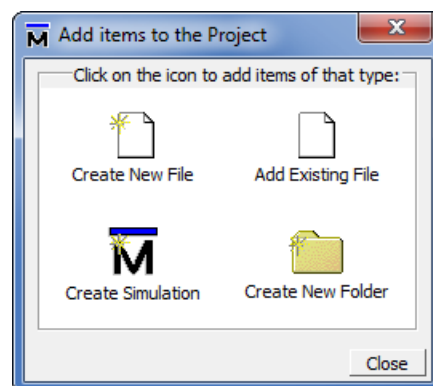


Figure 9. Add a file to project window.

The window in Figure 9 gives several options to add files to the project, including creating new files and directories, or adding existing files. Since the file for this tutorial exists, click **Add Existing File** and select *serial.v* file. Once the file is added to the project, it will appear in the Project tab on the left-hand side of the screen, as shown in Figure 10.

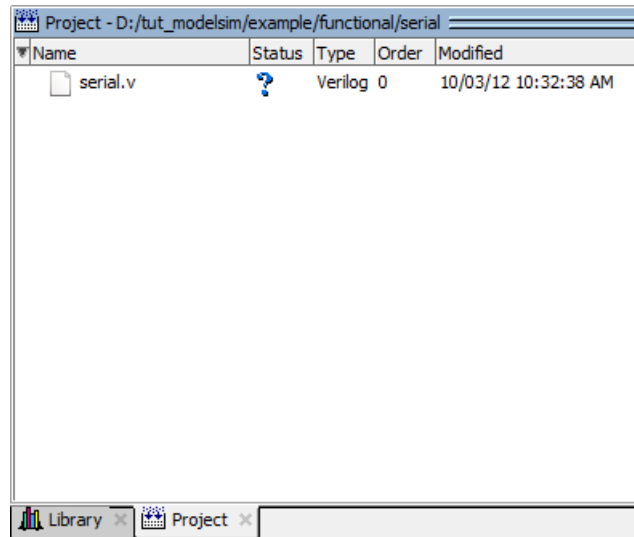


Figure 10. Workspace window after the project is created.

Now that all design files have been included in the project, click **Close** to close the window in Figure 9.

4.2 Compiling a Project

Once the project has been created, it is necessary to compile it. Compilation in ModelSim checks if the project files are correct and creates intermediate data that will be used during simulation. To perform compilation, select **Compile All** from the **Compile** menu. When the compilation is successful, a green check mark will appear to the right of the *serial.v* file in the Project tab.

4.3 Simulation

To begin a simulation of the design, the software needs to be put in simulation mode. To do this, select **Simulate > Start Simulation...**. The window in Figure 11 will appear.

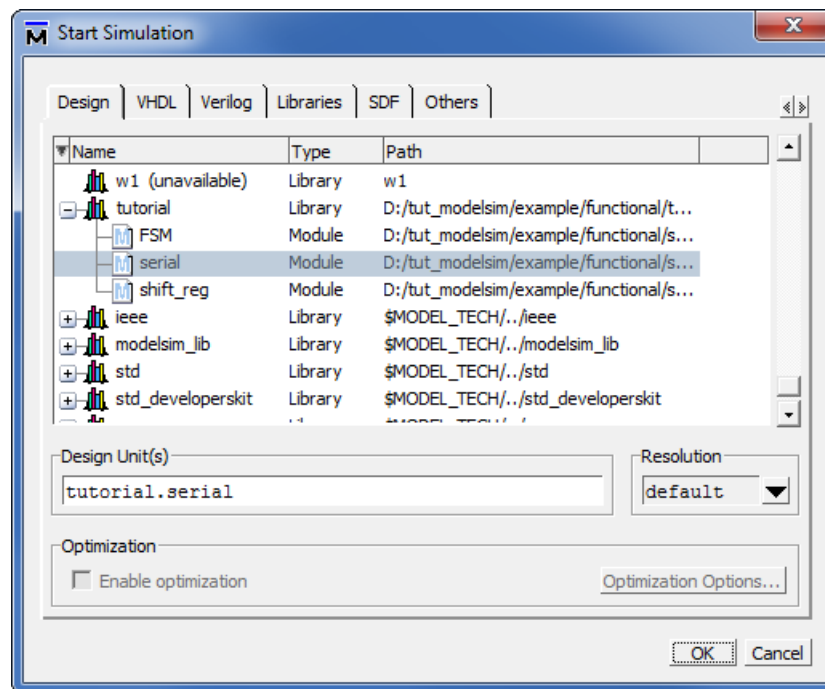


Figure 11. Start simulation mode in ModelSim.

The window to start simulation consists of many tabs. These include a **Design** tab that lists designs available for simulation, **VHDL** and **Verilog** tabs to specify language-specific options, a **Libraries** tab to include any additional libraries, and timing and other options in the remaining two tabs. For the purposes of the functional simulation, we only need to look at the **Design** tab.

In the **Design** tab you will see a list of libraries and modules you can simulate. In this tutorial, we want to simulate a module called *serial*, described in *serial.v* file. To select this module, scroll down and locate the *tutorial* library and click on the plus (+) sign. You will see three modules available for simulation: *FSM*, *serial*, and *shift_reg*. Select the *serial* module, as shown in Figure 11 and click **OK** to begin simulation.

When you click **OK**, ModelSim will begin loading the selected libraries and preparing to simulate the circuit. For the example in this tutorial, the preparation should complete quickly. Once ModelSim is ready to simulate your design, you will notice that several new tabs on the left-hand side of the screen and a new **Objects** window have appeared, as shown in Figure 12.

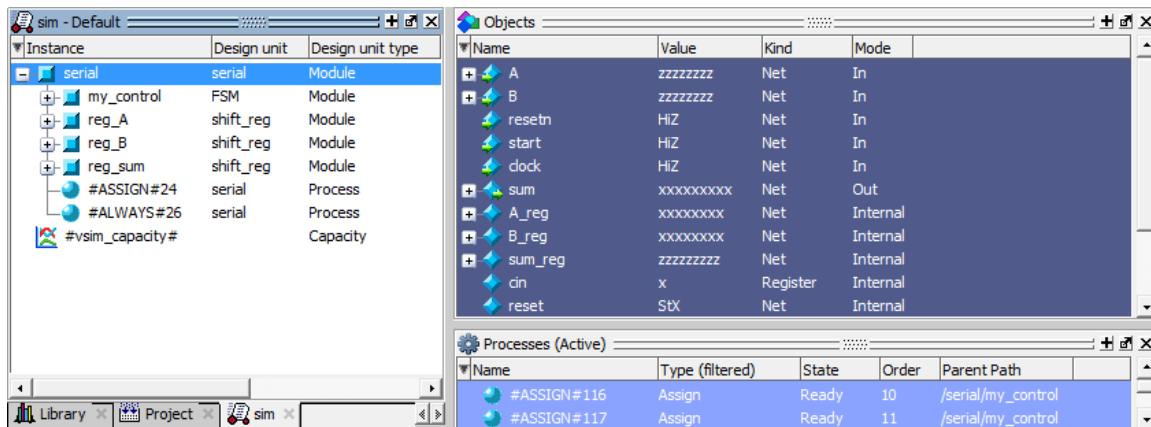


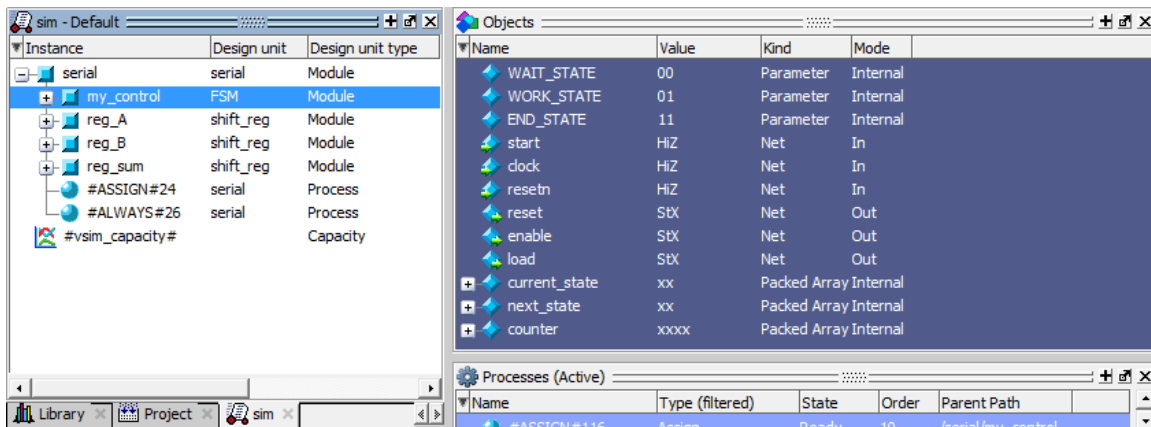
Figure 12. New displays in the simulation mode.

A key new tab on the left-hand side is the **sim** tab. It contains a hierarchical display of design units in your circuit in a form of a table. The columns of the table include the instance name, design unit and design unit type names. The rows of the table take a form of an expandable tree. The tree is rooted in the top-level entity called *serial*. Each module instance has a plus (+) sign next to its name to indicate it can be expanded to allow users to examine the contents of that module instance.

Expanding the top-level entity in this view gives a list of modules and/or constructs within it. For example, in Figure 12 the top-level entity *serial* is shown to contain an instance of the FSM module, called *my_control*, three instances of a *shift_reg* module, one assign statement and an always block. Double-clicking on any of the constructs will cause ModelSim to open a source file and locate the given construct within it. Double-clicking on a module instance will open a source file and point to the description of the module in the source file.

In addition to showing modules and/or constructs, the **sim** tab can be used to locate signals for simulation. Notice that when the *serial* module is highlighted, a list of signals (inputs, outputs, and local wires) is shown in the **Objects** window. The signals are displayed as a table with four columns: name, value, kind, and mode. The name of a signal may be preceded by a plus (+) sign to indicate that it is a bus. The top-level entity comprises signals *A*, *B*, *resetsn*, *start*, and *clock* as inputs, a *sum* output and a number of internal signals.

We can also locate signals inside of module instances in the design. To do this, highlight a module whose signals you wish to see in the **Objects** window. For example, to see the signals in the *my_control* instance of the FSM module, highlight the *my_control* instance in the **sim** tab. This will give a list of signals inside of the instance as shown in Figure 13.

Figure 13. Expanded *my_control* instance.

Using the *sim* tab and the *Objects* window we can select signals for simulation. To add a signal to simulation, right-click on the signal name in the *Objects* window and select **Add > To Wave > Selected Signals** from the pop-up menu. Using this method, add signals *A*, *B*, *resetrn*, *start*, *clock*, *sum*, and *current_state* to the simulation. When you do so, a waveform window will appear in the work area. Once you have added these signals to the simulation, press the **Undock** button in the top-right corner of the waveform window to make it a separate window, as shown in Figure 14.

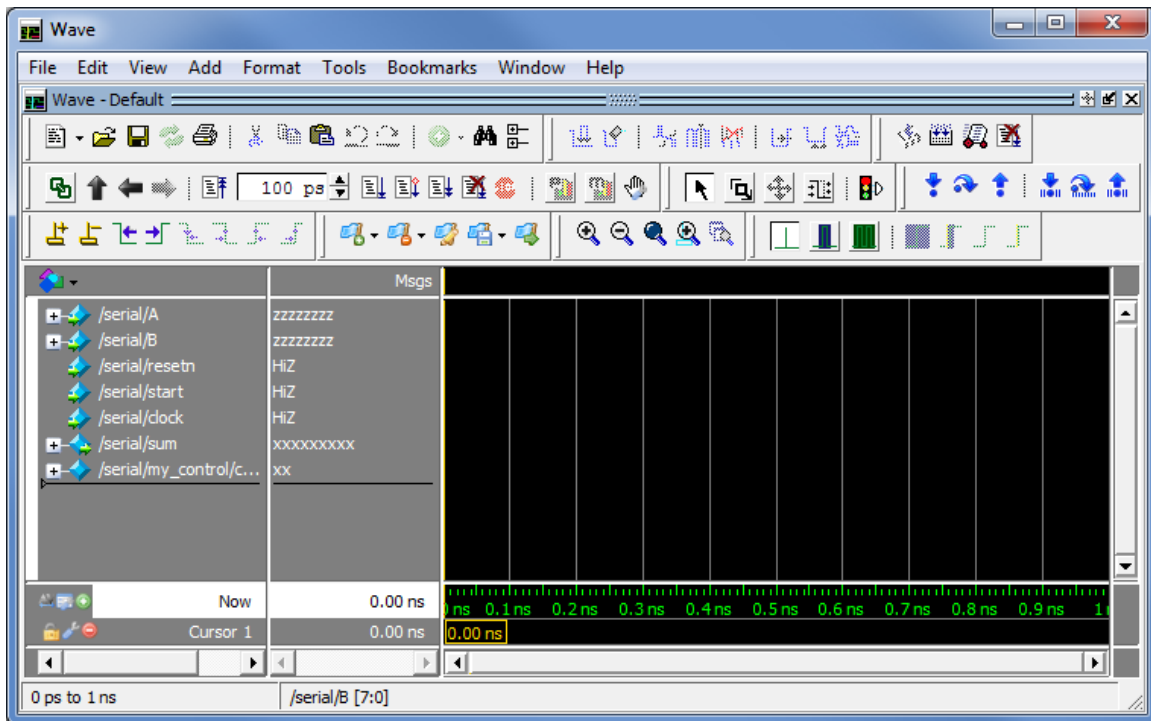


Figure 14. A simulation window.

Before we begin simulating the circuit, there is one more useful feature worth noting. It is the ability to combine signals and create aliases. It is useful when signals of interest are not named as well as they should be, or the given names are inconvenient for the purposes of simulation. In this example, we rename the *start* signal to *go* by highlighting the *start* signal and selecting Tools > Combine Signals.... The window in Figure 15 will appear.

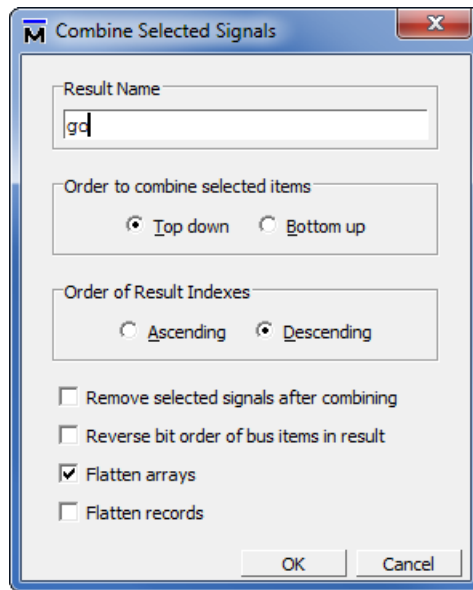


Figure 15. Combine signals window.

In the text field labeled **Result name** type *go* and press the **OK** button. This will cause a new signal to appear in the simulation window. It will be named *go*, but it will have an orange diamond next to its name to indicate that it is an alias. Once the *go* alias is created, the original *start* input is no longer needed in the simulation window, so remove it by highlighting it and pressing the delete key. Your simulation window should now look as in Figure 16.

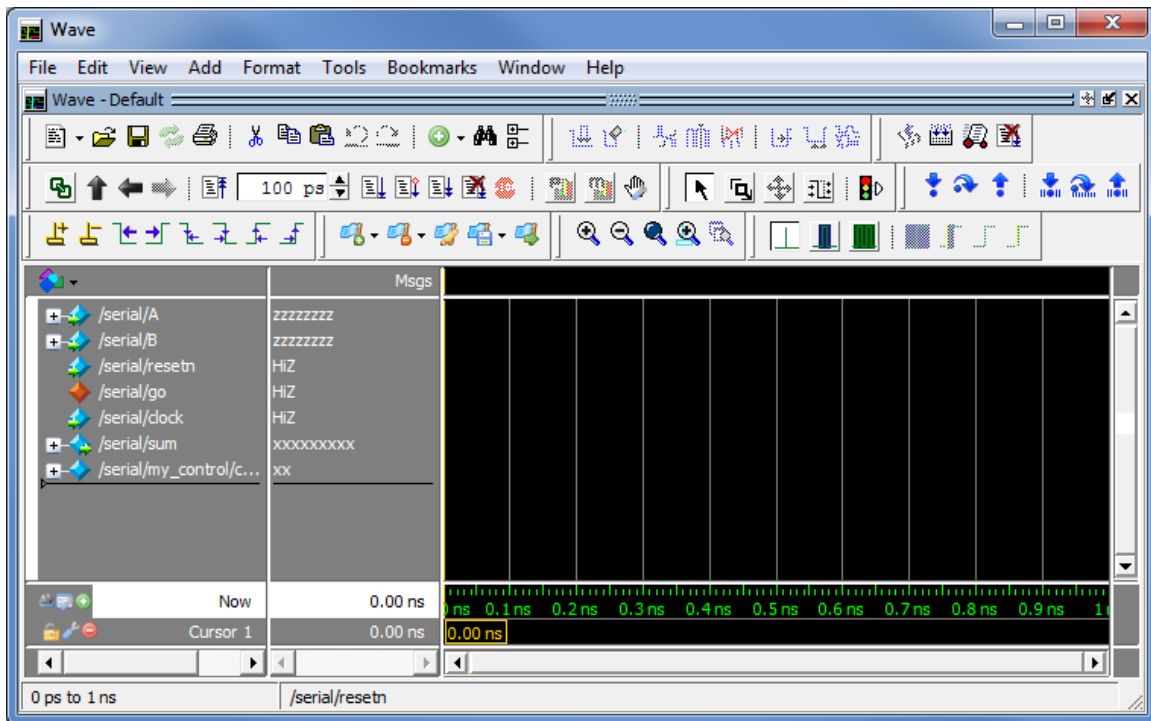


Figure 16. Simulation window with aliased signals.

Now that we set up a set of signals to observe we can begin simulating the circuit. There are two ways to run a simulation in ModelSim: manually or by using scripts. A manual simulation allows users to apply inputs and advance the simulation time to see the results of the simulation in a step-by-step fashion. A scripted simulation allows the user to create a script where the sequence of input stimuli are defined in a file. ModelSim can read the file and apply input stimuli to appropriate signals and then run the simulation from beginning to end, displaying results only when the simulation is completed. In this tutorial, we perform the simulation manually.

In this simulation, we use a clock with a 100 ps period. At every negative edge of the clock we assign new values to circuit inputs to see how the circuit behaves. To set the clock period, right-click on the *clock* signal and select *Clock...* from the pop-up menu. In the window that appears, set the clock period to 100 ps and the first edge to be the falling edge, as shown in Figure 17. Then click OK.

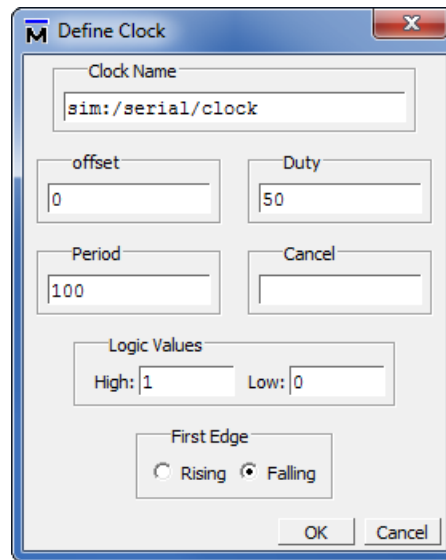


Figure 17. Set the clock period.

We begin the simulation by resetting the circuit. To reset the circuit, set the *resetn* signal low by right-clicking on it and selecting the **Force...** option from the pop-up menu. In the window that appears, set **Value** to 0 and click **OK**. In a similar manner, set the value of the *go* signal to 0. Now that the initial values for some of the signals are set, we can perform the first step of the simulation. To do this, locate the toolbar buttons shown in Figure 18.

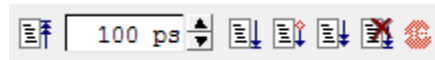


Figure 18. Simulation control buttons on the toolbar.

The toolbar buttons shown in Figure 18 are used to step through the simulation. The left-most button is the restart button, which causes the simulation window to be cleared and the simulation to be restarted. The text field, shown with a 100 ps string inside it, defines the amount of time that the simulation should run for when the **Run** button (to the right of the text field) is pressed. The remaining three buttons, **Continue**, **Run -All** and **Break**, can be used to resume, start and interrupt a simulation, respectively. We will not need them in this tutorial.

To run a simulation for 100 ps, set the value in the text field to 100 ps and press the **Run** button. After the simulation run for 100 ps completes, you will see the state of the circuit as shown in Figure 19. You can change the time scale of your waveform by going to **View > Zoom > Zoom Range....** Change the end time to 1200 ps and press **OK**.

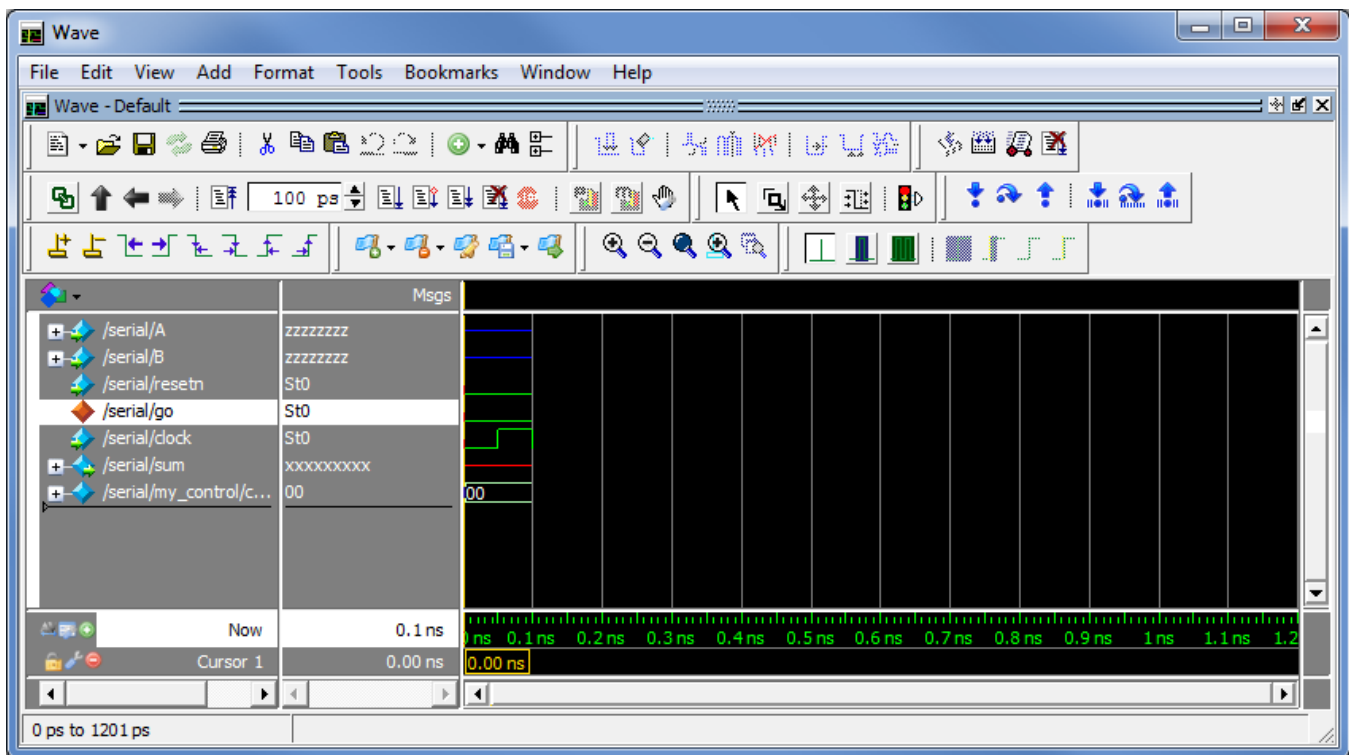
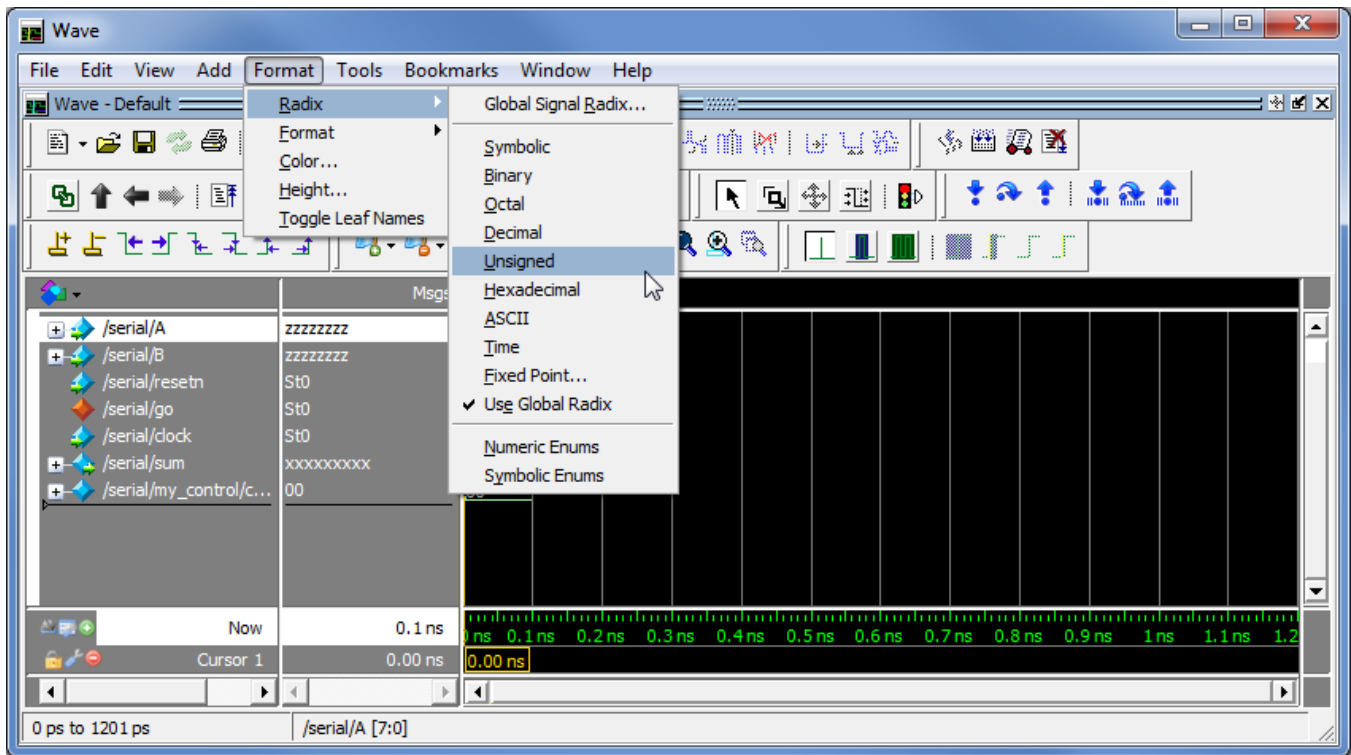


Figure 19. Simulation results after 100 ps.

In the figure, each signal has a logic state. The first two signals, *A* and *B*, are assigned a value between 0 and 1 in a blue color. This value indicates high impedance, and means that these signals are not driven to any logic state. The *go* and *resetn* signals are at a logic 0 value thereby resetting the circuit. The *clock* signal toggles state every 50 ps, starting with a falling edge at time 0, a rising edge at time 50 ps and another falling edge at 100 ps.

Now that the circuit is reset, we can begin testing to see if it operates correctly for desired inputs. To test the serial adder we will add numbers 143 and 57, which should result in a sum of 200. We can set *A* and *B* to 143 and 57, respectively, using decimal notation. To specify a value for *A* in decimal, right-click on it, and choose **Force...** from the pop-up menu. Then, in the *Value* field put 10#143. The 10# prefix indicates that the value that follows is specified in decimal. Similarly, set the *Value* field of *B* to 10#57.

To see the decimal, rather than binary, values of buses in the waveform window we need to change the *Radix* of *A* and *B* to *unsigned*. To change the radix of these signals, highlight them in the simulation window and select **Format > Radix > Unsigned**, as shown in Figure 20. Change the radix of the *sum* signal to *unsigned* as well.

Figure 20. Changing the radix of *A*, *B* and *sum* signals.

Now that inputs *A* and *B* are specified, set *resetn* to 1 to stop the circuit from resetting. Then set *go* to 1 to begin serial addition, and press the Run button to run the simulation for another 100 ps. The output should be as illustrated in Figure 21. Notice that the values of inputs *A* and *B* are shown in decimal as is the *sum*. The circuit also recognized a *go* signal and moved to state 01 to begin computing the sum of the two inputs.

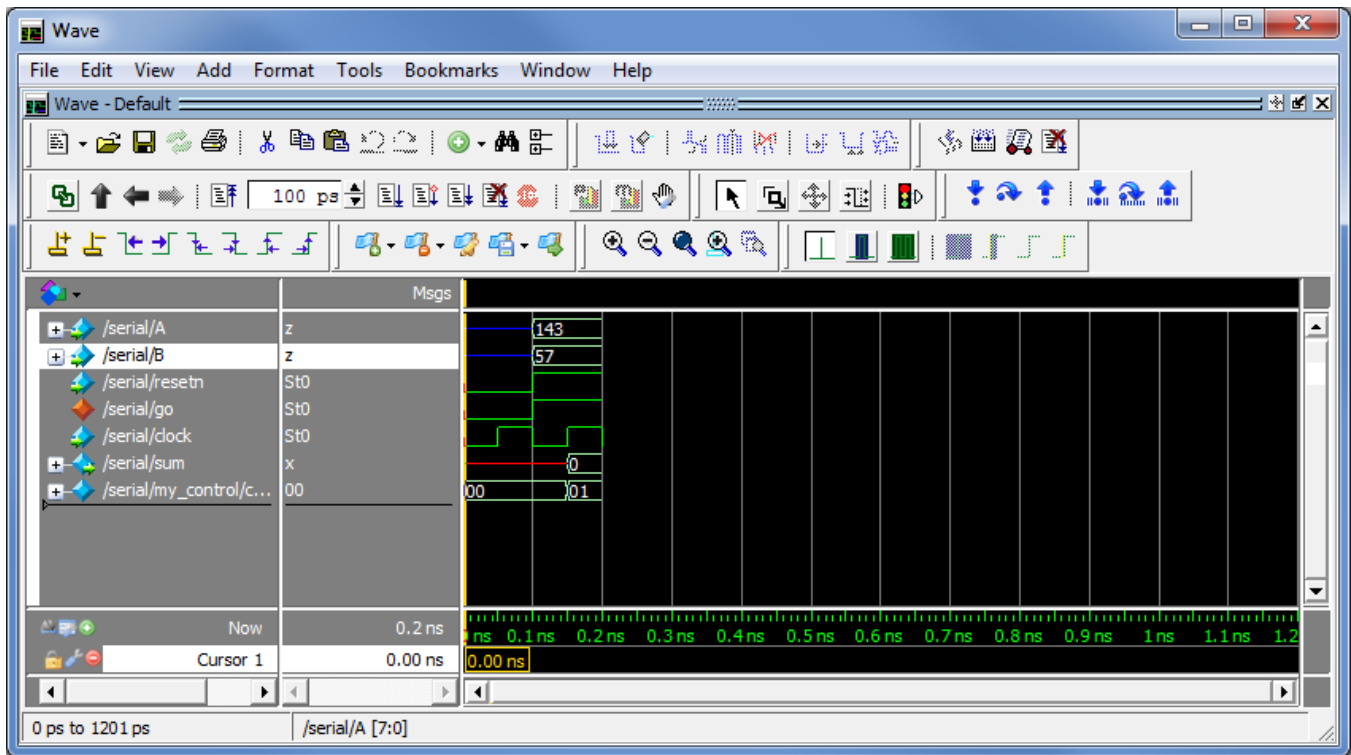


Figure 21. Simulation results after 200 ps.

To complete the operation, the circuit will require 9 clock cycles. To fast forward the simulation to see the result, specify 900 ps in the text field next to the run button, and press the run button. This brings the simulation to time 1100 ps, at which point a result of summation is shown on the `sum` signal, as illustrated in Figure 22.

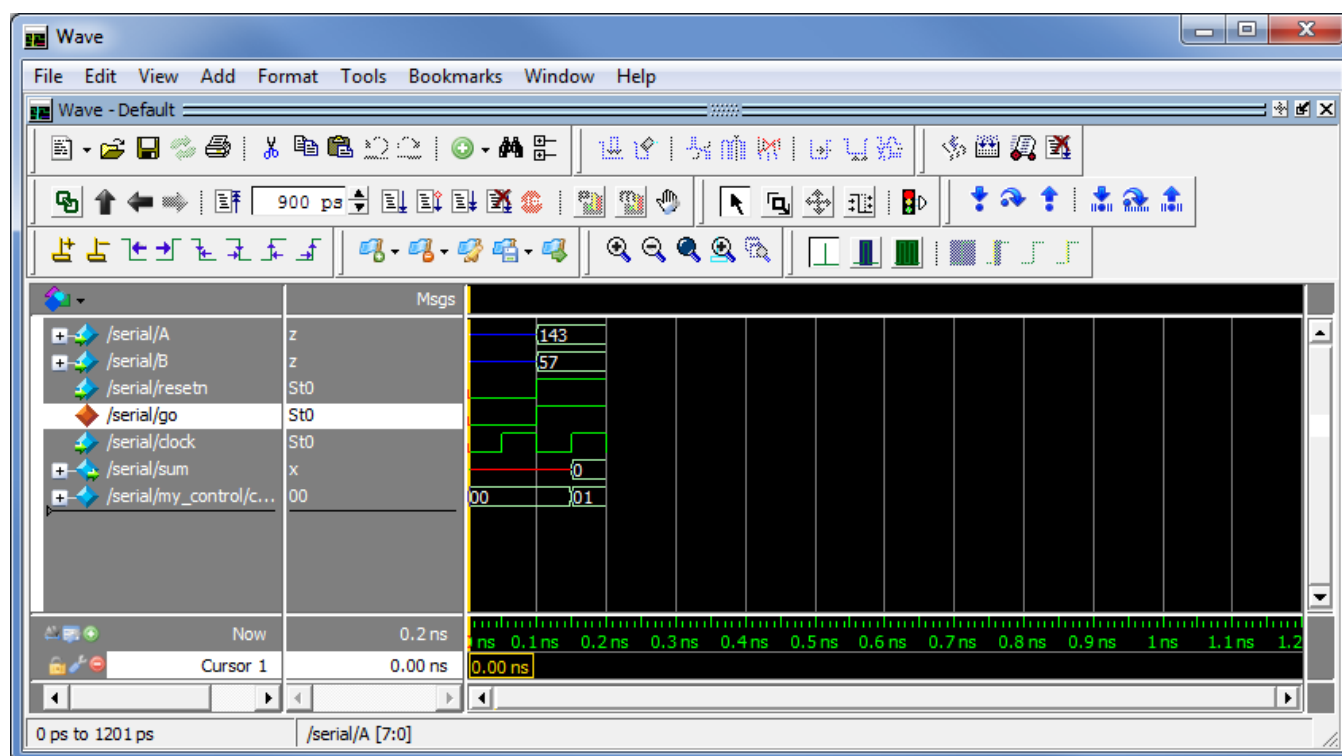


Figure 22. Simulation results after 1100 ps.

We can see that the result is correct and the finite state machine controlling the serial adder entered state 11, in which it awaits the *go* signal to become 0. Once we set the *go* signal to 0 and advance the simulation by 100 ps, the circuit will enter state 00 and await a new set of inputs for addition. The simulation result after 1200 ps is shown in Figure 23.

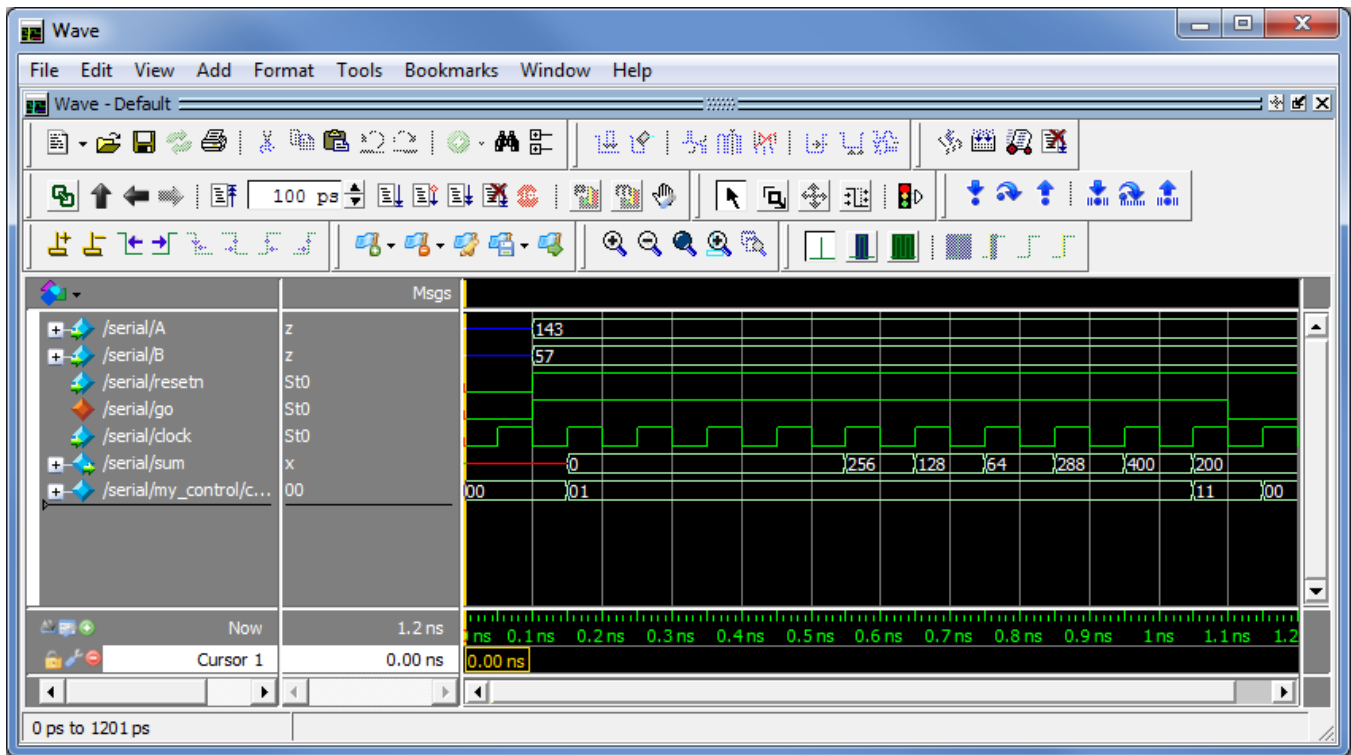


Figure 23. Simulation results after 1200 ps.

At this point, we can begin the simulation for a new set of inputs as needed, repeating the steps described above. We can also restart the simulation by pressing the restart button to begin again from time 0.

By using the functional simulation we have shown that the *serial.v* file contains an accurate Verilog HDL description of a serial adder. However, this simulation did not verify if the circuit implemented on an FPGA is correct. This is because we did not use a synthesized, placed and routed circuit as input to the simulator. The correctness of the implementation, including timing constraints can be verified using timing simulation.

5 Timing Simulation with ModelSim

Timing simulation is an enhanced simulation, where the logical functionality of a design is tested in the presence of delays. Any change in logic state of a wire will take as much time as it would on a real device. This forces the inputs to the simulation be realistic not only in terms of input values and the sequence of inputs, but also the time when the inputs are applied to the circuit.

For example, in the previous section we simulated the sample design and used a clock period of 100 ps. This clock period is shorter than the minimum clock period for this design, and hence the timing simulation would fail to produce the correct result. To obtain the correct result, we have to account for delays when running the simulation and use a clock frequency for which the circuit operates correctly.

For Altera FPGA-based designs the delay information is available after the design is synthesized, placed and routed, and is generated by Quartus II CAD software. The project for this part of the tutorial has been created for you in the *example/timing* subdirectory.

5.1 Setting up a Quartus II Project for Timing Simulation with ModelSim

To perform timing simulation we need to set up Quartus II software to generate the necessary delay information for ModelSim by setting up EDA Tools for simulation in the Quartus II project. To set up EDA Tools for simulation, open the Quartus II project in *example/timing* subdirectory, and select **Assignment > Settings....** A window shown in Figure 24 will appear.

The window in the figure consists of a list on the left-hand side to select the settings category and a window area on the right-hand side that displays the settings for a given category. Select **Simulation** from the *EDA Tool Settings* category to see the screen shown on the right-hand side of Figure 24.

The right-hand side of the figure contains the tool name at the top, EDA Netlist Writer settings in the middle, and NativeLink settings at the bottom. The tool name is a drop-down list containing the names of simulation tools for which Quartus II can produce a netlist with timing information automatically. This list contains many well-known simulation tools, including ModelSim. From the drop-down list select **ModelSim-Altera**.

Once a simulation tool is selected, *EDA Netlist Writer settings* become available. These settings configure Quartus II to produce input for the simulation tool. Quartus II will use these parameters to describe an implemented design using a given HDL language, and annotate it with delay information obtained after compilation. The settings we can define are the HDL language, simulation time scale that defines time step size for the simulator to use, the location where the writer saves design and delay information, and others. Set these settings to match those shown in Figure 24 and click **OK**.

With the EDA Tools Settings specified, we can proceed to compile the project in Quartus II. The compilation process synthesizes, places, and routes the design, and performs timing analysis. Then it stores the compilation result in the simulation directory for ModelSim to use. Take a moment to examine the files generated for simulation using a text editor. The two main files are *serial.vo*, and *serial_v.sdo*.

The *serial.vo* file is a Verilog file for the design. The file looks close to the original Verilog file, except that the design now contains a wide array of modules with a *cycloneii_* prefix. These modules describe resources on an Altera Cyclone II FPGA, on which the design was implemented using lookup tables, flip-flops, wires and I/O ports. The list of delays for each module instance in the design is described in the *serial_v.sdo* file.

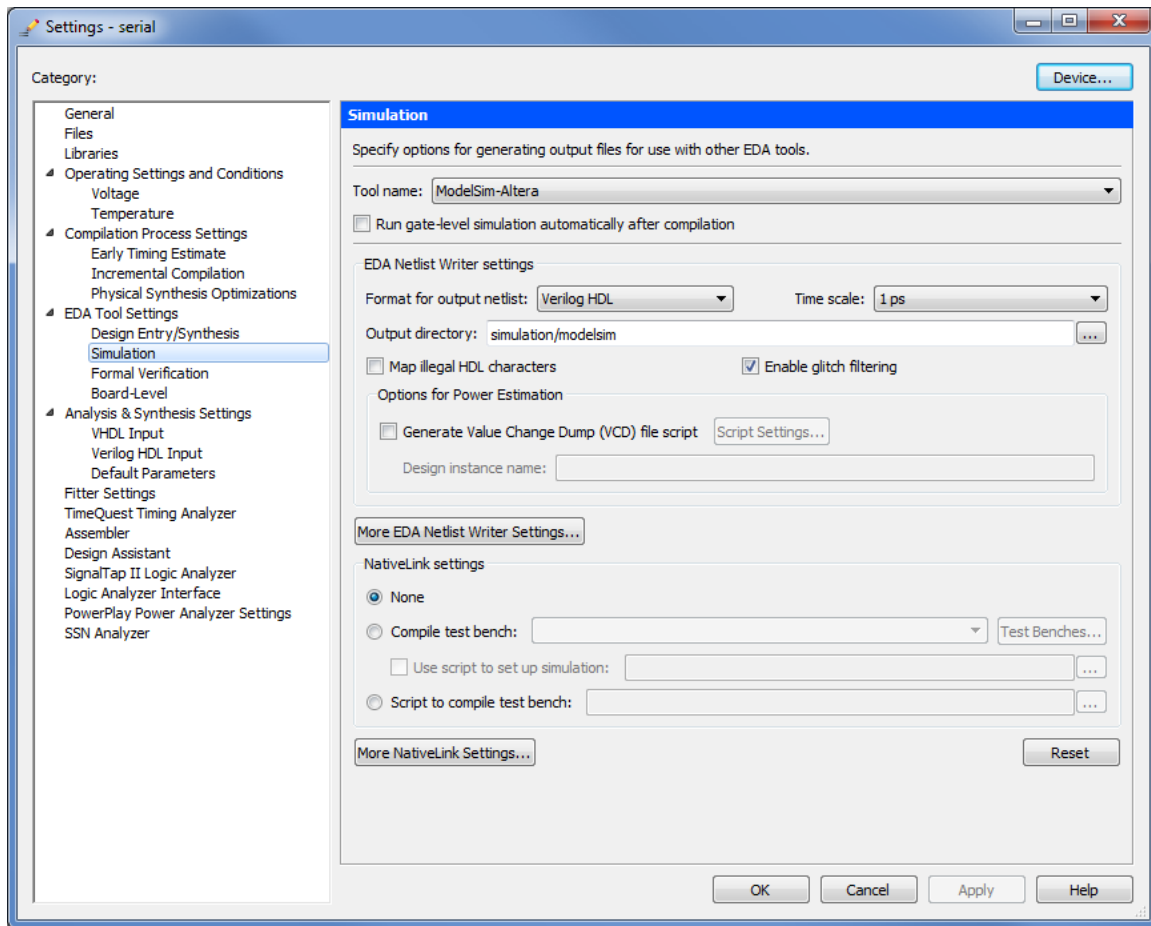


Figure 24. Quartus II EDA simulation tool settings.

5.2 Running a Timing Simulation

To simulate the design using timing simulation we must create a ModelSim project. The steps are the same as in the previous section; however, the project is located in the *example/timing/simulation/modelsim* subdirectory, and the source file is *serial.vo*. We do not need to include the *serial_v.sdo* file in the project, because a reference to it is included in the *serial.vo* file. Once you added the source file to the project, compile it by selecting **Compile > Compile All**.

The next step in the simulation procedure is to place the ModelSim software in simulation mode. In the previous section, we did this by selecting **Simulate > Start Simulation...**, and specifying the project name. To run a timing simulation there is an additional step required to include the Altera Cyclone II device library in the simulation. This library contains information about the logical operation of modules with the *cycloneii_* prefix. To include the Cyclone II library in the project, select **Simulate > Start Simulation...** and select the **Libraries** tab as shown in Figure 25.

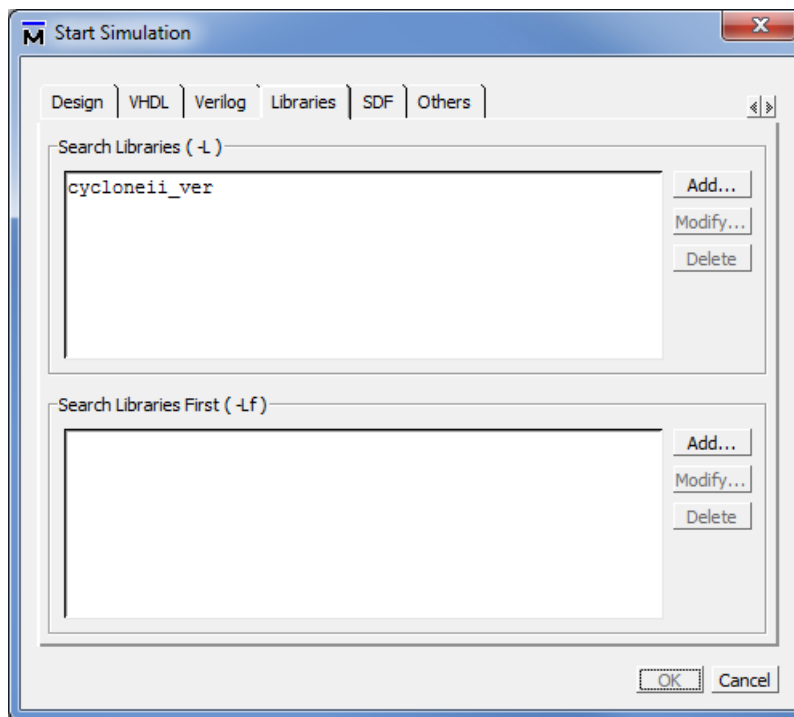


Figure 25. Including Altera Cyclone II library in ModelSim project.

The Altera Cyclone II library is located in the *altera/verilog/cycloneii* directory in the ModelSim-Altera software. To add this library to your project, select **Add...** and choose *cycloneii_ver* from the dropdown list. Then, click on the **Design** tab, select your project for simulation (tutorial > serial), and click **OK**.

When the ModelSim software enters simulation mode, you will see a significant difference in the contents of the workspace tabs on the left-hand side of the window as compared to when you ran the functional simulation. In particular, notice the **sim** tab and the **Objects** window shown in Figure 26. The list of modules in the **sim** tab is larger, and the objects window contains more signals. This is due to the fact that the design is constructed using components on an FPGA and is more detailed in comparison to an abstract description we used in the previous section of the tutorial.

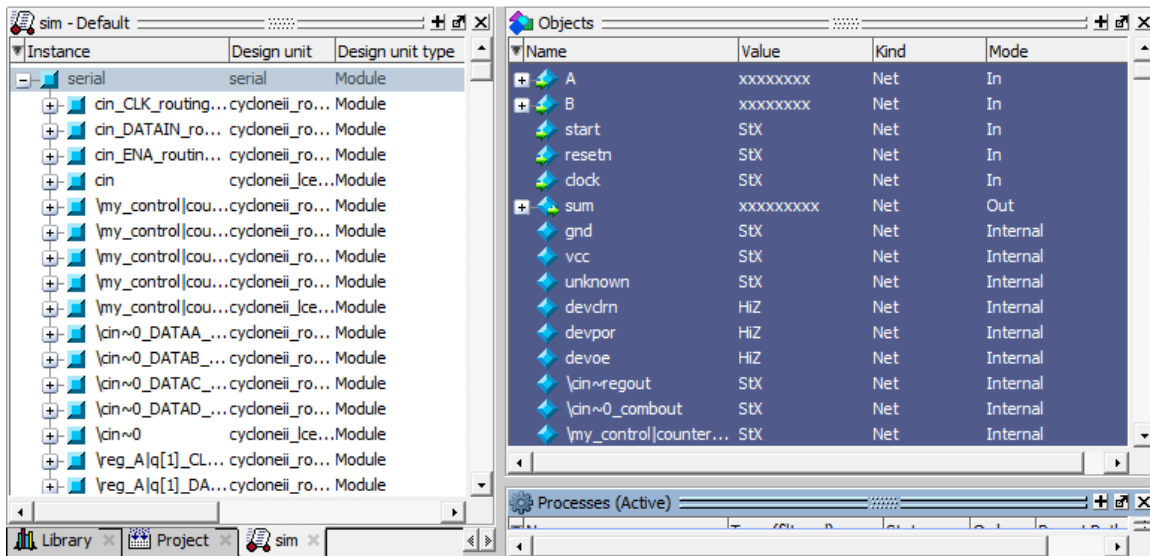


Figure 26. Workspace tabs and Objects window for timing simulation.

We simulate the circuit by creating a waveform that includes signals *sum*, *A*, *B*, *start* (*go*), and *resetrn* aliases as before. In addition, we include the *clock*, *reg_sumlq*, *reg_Alq*, and *reg_Blq* signals from the Objects window. Signals *reg_Alq* and *reg_Blq* are registers that store *A* and *B* at the positive edge of the clock. The *reg_sumlq* signal is a register that stores the resulting sum.

Begin the simulation by resetting the circuit. To do this, set the *go* and *resetrn* signals to 0. Also, set the *clock* input to have a period of 20 ns, whose first edge is a falling edge. To run the simulation, set the simulation step to 20 ns and press the Run button. The simulation result is shown in Figure 27.

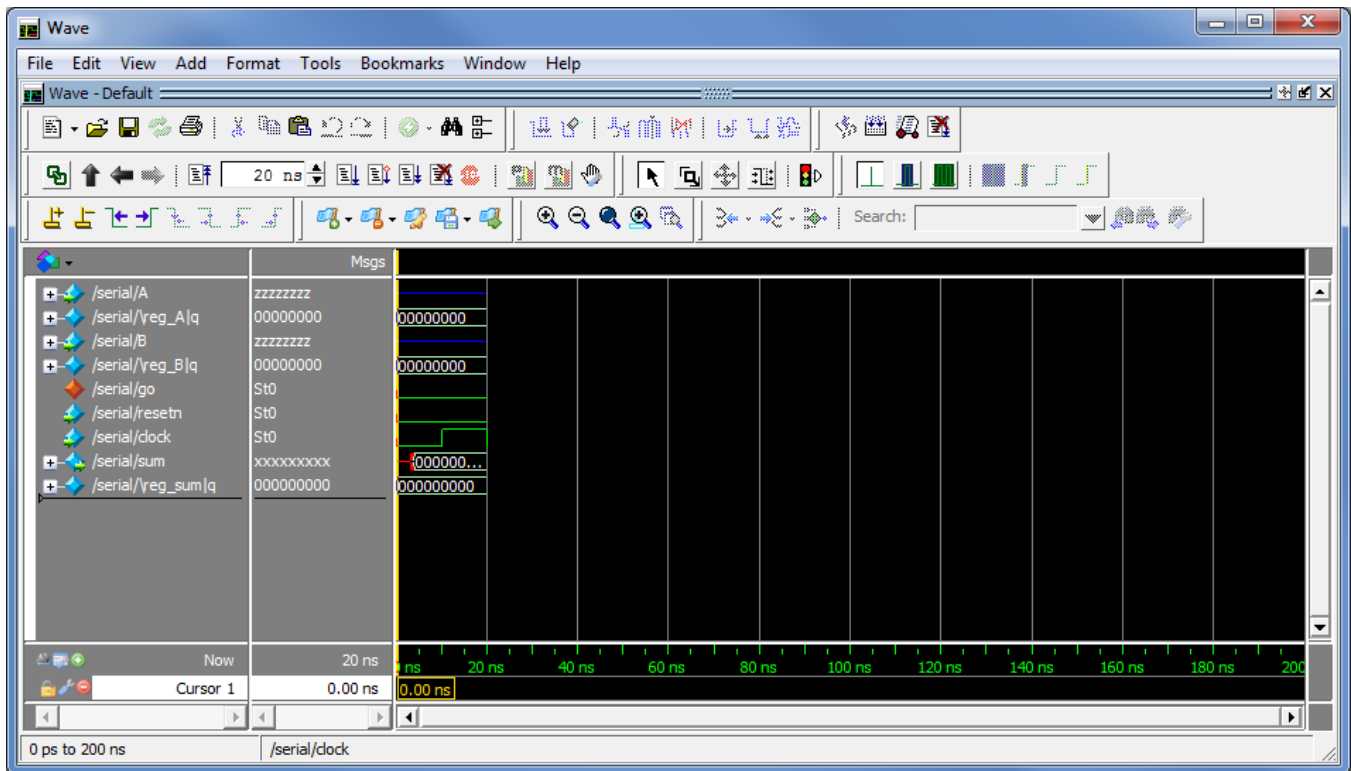


Figure 27. Timing Simulation after 20 ns.

To proceed with the simulation deassert the *resetn* signal by setting it to 1, and apply data to inputs *A* and *B*. Set them to 143 and 57, and assign a value of 1 to the *go* input as described in the Functional Simulation section of the tutorial. Then run the simulation for a period of 20 ns, by pressing the Run button. The simulation result is shown in Figure 28. Remember to change the radix of *A*, *B*, *sum*, and their corresponding registers to *unsigned*.



When we zoom in on the time when registers *reg_Alq* and *reg_Blq* change value, we see the register values change as shown in Figure 29. In the figure, register *reg_Blq* stabilizes on a value of 57 at time 32946 ps. This is 2946 ps after the positive edge of the clock appeared at the *clock* input. Part of the difference in times between the clock edge and the change of data in register *reg_Blq* comes from the fact that the clock signal must travel from the input pin on the FPGA device to the registers. The other part of the time is due to the clock-to-Q time of the register, which is the time it takes for a register to change output after it sees a positive edge on the clock input.

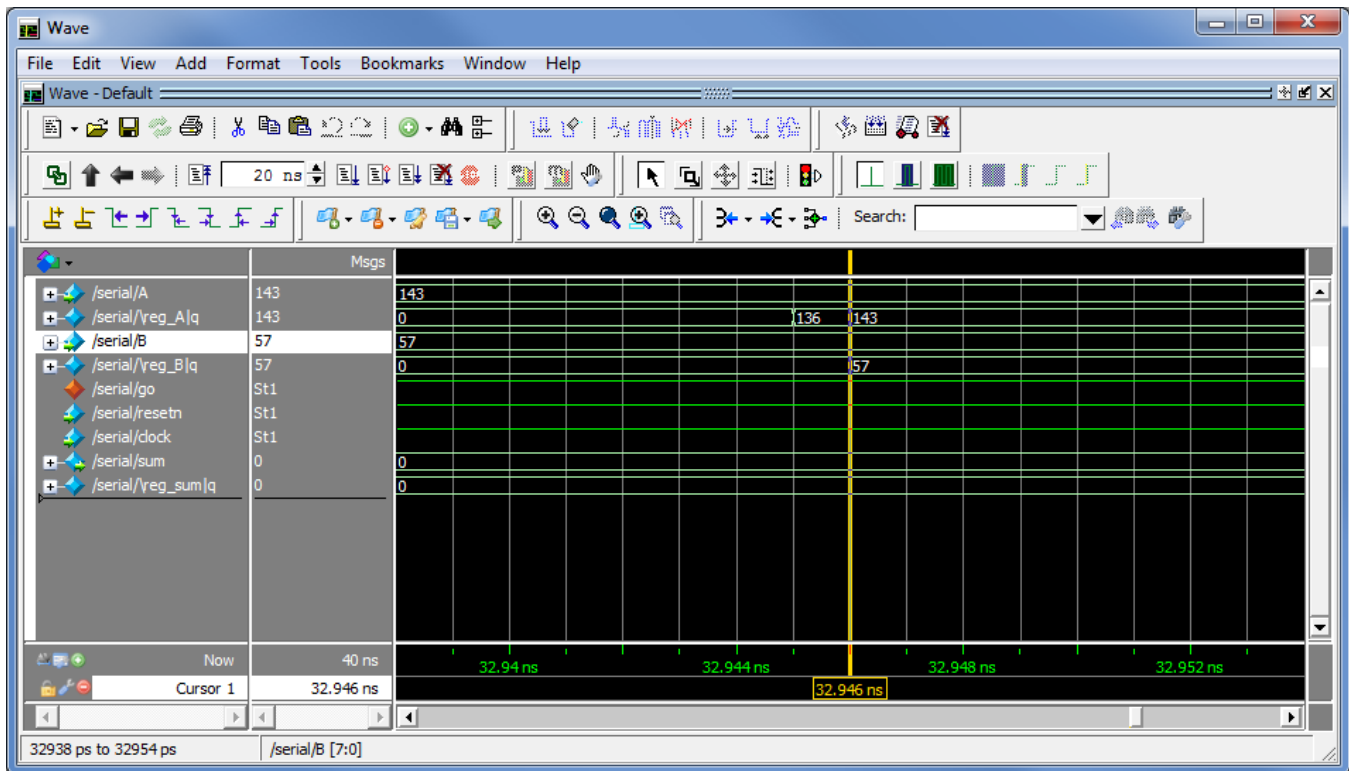


Figure 29. Zoomed-in Timing Simulation after 40ns.

Another interesting aspect of the timing simulation can also be observed in Figure 29. Notice that register *reg_Alq* first changes value to 136 and a few picoseconds later assumes the value 143. This is because the *clock* signal does not get to every flip-flop at exactly the same time - this is called *clock skew*.

6 Concluding Remarks

This tutorial discussed the basic use of ModelSim simulator. We demonstrated how to perform a functional simulation using a user-written Verilog code, as well as a detailed timing simulation. For the timing simulation, we presented a simple method to generate design description using Altera Quartus II CAD software, which includes the low-level design details and circuit delays.

There are more advanced options for simulation available in ModelSim software. They can help automate and speed up the simulation of larger more complex designs. These features are covered in the tutorials provided with the ModelSim tool, and are beyond the scope of this introductory tutorial.

Copyright ©1991-2012 Altera Corporation. All rights reserved. Altera, The Programmable Solutions Company, the stylized Altera logo, specific device designations, and all other words and logos that are identified as trademarks and/or service marks are, unless noted otherwise, the trademarks and service marks of Altera Corporation in the U.S. and other countries. All other product or service names are the property of their respective holders. Altera products are protected under numerous U.S. and foreign patents and pending applications, mask work rights, and copyrights. Altera warrants performance of its semiconductor products to current specifications in accordance with Altera's standard warranty, but reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera Corporation. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.

This document is being provided on an "as-is" basis and as an accommodation and therefore all warranties, representations or guarantees of any kind (whether express, implied or statutory) including, without limitation, warranties of merchantability, non-infringement, or fitness for a particular purpose, are specifically disclaimed.